

Задание 4

Произведение матриц с помощью интринсик

Отчет

Загоруйко Т.Е.

2024

1. Постановка задачи

Требуется реализовать векторизованную версию матричного умножения с помощью интринсик используемой целевой архитектуры (AVX) при условии таких требований, как:

- Хранение всех матриц предполагается в едином порядке (все в row-major или в col-major).

2. Формат командной строки

./task_avx

3. Спецификация системы

Вычисления производились внутри виртуальной машины Oracle VM VirtualBox, ОС: Ubuntu (64-bit). По результатам команды `lscpu`:

```
tatiana@Ubuntu-18:~/1Study/tspp_homework/sem04$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:   0-3
Thread(s) per core:    1
Core(s) per socket:    4
Socket(s):              1
NUMA node(s):          1
Vendor ID:              GenuineIntel
CPU family:             6
Model:                 60
Model name:             Intel(R) Core(TM) i7-4702MQ CPU @ 2.20GHz
Stepping:               3
CPU MHz:                2194.942
BogoMIPS:               4389.88
L1d cache:              32K
L1i cache:              32K
L2 cache:               256K
L3 cache:               6144K
NUMA node0 CPU(s):     0-3
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr
sse sse2 ht syscall nx rdtscp lm constant_tsc rep_good nopl xtopology nonstop_tsc cpuid pni pclmulqdq ssse3 cx
16 pcid sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx rdrand hypervisor lahf_lm abm invpcid_single pti fsgsb
ase bmi1 avx2 bmi2 invpcid arat md_clear flush_l1d
```

4. Результаты выполнения

Были реализованы две программы: с использованием интринсик `task_avx` и обычная реализация `task_seq`. Программы запускалась для 5 размерностей матриц (матрицы считаем квадратными): 16x16, 64x64, 128x128, 256x256, 512x512.

В программе использовались следующие интринсики AVX:

- **_mm256_loadu_pd**: используется для загрузки данных из памяти в регистры SIMD в формате 256 бит без учета выравнивания. Позволяет загружать данные из произвольных адресов, но может быть медленнее по сравнению с выровненными загрузками;
- **_mm256_set1_pd**: используется для инициализации вектора и заполнения его элементов одинаковым значением типа double;
- **_mm256_fmadd_pd**: используется для одновременного умножения двух векторов и сложения результата с третьим вектором;
- **_mm256_storeu_pd**: используется для сохранения данных из векторов в память, также работает с невыровненной областью памяти, как и первый интринсик.

Компиляция с интринсиками производилась следующей командой:

```
gcc -DDIM=<размерность_матрицы> -mavx2 -mfma -Wall -Wextra -g -O3  
task_avx.c -o task_avx
```

Компиляция обычной реализации производилась следующей командой:

```
gcc -DDIM=<размерность_матрицы> -Wall -Wextra -g -O3 task_seq.c -o  
task_seq
```

На графике ниже представлено сравнение работы task_avx и task_seq. Можно заметить, что программа task_seq отработала медленнее на всех размерностях матриц A и B.

